

改进后的完整软件过程定义文档

一、项目概述

1.1 项目背景

本项目选取“ORB-SLAM3单目SLAM系统在Windows平台下的部署与实验”作为研究对象。

ORB-SLAM3是一种基于视觉特征的同步定位与地图构建（Simultaneous Localization and Mapping, SLAM）系统，广泛应用于机器人导航、自动驾驶、增强现实以及三维重建等领域。相比前代系统，ORB-SLAM3在鲁棒性、精度以及多地图管理等方面具有明显优势，因此成为当前视觉SLAM研究的重要平台之一。

本项目的主要任务是在Windows平台完成ORB-SLAM3系统的编译、部署与运行，并利用公开数据集开展单目SLAM实验，对实验结果进行分析与验证。

1.2 项目目标

本项目的主要目标包括：

- 完成ORB-SLAM3在Windows平台下的部署
- 配置并整合相关依赖库
- 成功运行单目SLAM实验
- 输出轨迹与地图结果
- 对实验结果进行分析与验证
- 探索AI技术在软件开发过程中的应用价值

二、软件过程定义依据

本项目的软件过程定义主要来源于以下三个方面。

2.1 ISO/IEC 12207标准

ISO/IEC 12207定义了软件生命周期过程，为软件开发提供了完整的过程框架。

本项目主要参考其中的：

- 需求分析过程
- 设计过程
- 实现过程
- 测试过程
- 验证过程
- 管理过程

并结合项目特点进行合理剪裁。

2.2 软件过程与管理课程内容

通过课程学习，掌握了软件过程的层次结构、生命周期模型（如瀑布模型与迭代模型）以及软件过程分类方法等核心内容。这些理论知识为本项目的软件过程设计提供了理论基础，使软件过程划分更加规范和系统。

2.3 AI赋能软件工程思想

近年来，生成式人工智能技术快速发展，大模型已经广泛应用于软件工程领域。

本项目引入以下AI工具：

- ChatGPT
- Gemini
- Cursor

利用AI辅助完成需求分析、系统设计、代码开发、测试设计以及结果分析等活动，构建智能化软件过程体系。

三、改进后的软件过程总体框架

在原有软件过程基础上，引入AI辅助活动，形成新的软件过程体系。

改进后的软件过程包括：

1. AI辅助需求分析过程
2. AI辅助系统设计过程
3. AI辅助实现过程
4. AI辅助测试过程
5. AI辅助验证分析过程
6. AI辅助项目管理过程

总体流程如下：

```
需求分析
↓
AI需求分析
↓
系统设计
↓
AI架构分析
↓
编码实现
↓
AI辅助开发
↓
测试验证
↓
AI测试设计
↓
结果分析
↓
AI辅助分析
↓
项目总结
```

该过程采用“人工决策、AI辅助、人机协同”的开发模式。

四、AI辅助需求分析过程

4.1 过程目标

明确项目目标、功能需求以及约束条件，为后续开发活动提供依据。

4.2 输入

项目总体目标：

完成ORB-SLAM3在Windows平台下的部署与实验验证。

4.3 过程活动

人工活动

开发人员负责：

- 分析课程任务要求；
- 明确实验目标
- 收集相关资料
- 提出需求描述

AI活动

利用ChatGPT辅助完成：

- 需求整理
- 功能需求生成
- 非功能需求分析
- 需求冲突检查
- 需求遗漏分析

4.4 输出

形成需求说明文档。

功能需求

- 系统能够正常运行
- 能够读取数据集
- 能够完成定位与建图
- 能够输出轨迹结果
- 能够保存实验结果

非功能需求

- 系统稳定运行
- 具备一定实时性
- 满足实验验证需求

五、AI辅助系统设计过程

5.1 过程目标

理解ORB-SLAM3整体架构并完成设计规划。

5.2 人工活动

开发人员负责：

- 阅读与ORB-SLAM3相关的文件
- 阅读源码
- 学习系统架构

5.3 AI活动

利用ChatGPT分析系统结构：

Tracking模块

负责图像跟踪与位姿估计。

Local Mapping模块

负责局部地图构建与优化。

Loop Closing模块

负责闭环检测与全局优化。

Atlas模块

负责多地图管理。

AI辅助生成：

- 模块说明
- 数据流分析
- UML结构描述
- 系统运行流程说明

5.4 输出

形成系统设计说明。

包括：

- 系统结构分析
- 模块关系说明
- 数据流描述
- 运行流程设计

六、AI辅助实现过程

6.1 过程目标

完成开发环境搭建与系统实现。

6.2 人工活动

开发人员负责：

- 安装Visual Studio
- 配置CMake
- 配置OpenCV
- 配置Eigen
- 配置其他依赖库

6.3 AI活动

利用ChatGPT和Cursor辅助开发。

主要应用于：

编译错误分析

例如：

- CMake配置错误
- OpenCV版本冲突
- 链接错误
- 依赖缺失问题

代码修改建议

例如：

- API适配
- Windows平台兼容
- 头文件调整
- 代码优化建议

6.4 输出

成功编译并运行的ORB-SLAM3系统。

七、AI辅助测试过程

7.1 过程目标

验证系统功能是否正确实现。

7.2 人工活动

开发人员负责：

- 执行测试
 - 记录结果
 - 分析缺陷
-

7.3 AI活动

利用ChatGPT自动生成测试方案。

功能测试

- 系统启动测试
- 数据集加载测试
- 地图构建测试
- 轨迹输出测试
- 特征点提取测试

异常测试

- 数据集路径错误测试
- 配置文件缺失测试
- 图像损坏测试
- OpenCV兼容性测试

性能测试

- CPU占用率测试
- 内存占用测试
- 实时性能测试

兼容性测试

- 不同数据集测试
 - 不同图像分辨率测试
 - 不同运行环境测试
-

7.4 输出

形成测试报告。

包括：

- 测试结果
- 问题记录
- 缺陷分析
- 修复建议

八、AI辅助验证分析过程

8.1 过程目标

验证系统是否达到预期实验目标。

8.2 人工活动

开发人员负责：

- 查看实验结果
- 分析轨迹
- 对比实验目标

8.3 AI活动

利用ChatGPT辅助分析：

- 轨迹误差
- 漂移现象
- 特征点丢失
- 数据质量影响
- 光照变化影响

8.4 输出

实验分析报告。

包括：

- 结果评价
- 问题分析
- 优化建议
- 实验结论

九、AI辅助项目管理过程

9.1 过程目标

保证项目按计划进行。

9.2 人工活动

- 制定开发计划
- 跟踪项目进度
- 管理任务执行情况

9.3 AI活动

利用AI辅助：

- 自动生成任务清单
- 自动生成周计划
- 自动生成项目总结
- 提供风险提示

9.4 输出

项目进度记录与总结报告。

十、新增角色定义

引入AI后，新增以下角色。

10.1 AI需求分析师

职责：

- 辅助整理用户需求
- 自动生成需求文档
- 识别需求冲突与遗漏

对应工具：

- ChatGPT

10.2 AI开发助手

职责：

- 辅助三方库编译与项目整体部署
- 错误分析
- 代码重构建议

对应工具：

- Cursor
 - ChatGPT
-

10.3 AI测试分析师

职责：

- 自动生成测试用例
- 分析测试结果
- 提供测试优化建议

对应工具：

- ChatGPT
-

10.4 AI知识助手

职责：

- 查询技术文档
- 提供问题解决方案
- 总结实验结果

对应工具：

- ChatGPT
 - Gemini
-

十一、过程质量保证

为了避免AI生成错误内容，需要建立质量保证机制。

11.1 人工审核机制

所有AI生成内容均需人工审核。

包括：

- 需求文档；
 - 设计文档；
 - 测试方案；
 - 实验分析报告。
-

11.2 双重验证机制

关键结论必须同时经过：

- AI分析；
- 人工确认。

确保结果准确可靠。

11.3 版本管理机制

采用Git进行：

- 代码管理；
- 文档管理；
- 修改记录管理。

保证开发过程可追踪。

十二、过程度量指标

为了评价AI引入后的效果，建立以下指标体系。

指标	说明
需求整理时间	完成需求分析所需时间
设计时间	完成系统设计所需时间
调试时间	问题修复耗时
测试用例数量	测试覆盖程度
缺陷发现率	测试发现问题能力
项目总周期	项目完成时间

AI改进效果评估

环节	改进前	改进后
需求分析	(约) 4小时	(约) 1小时
系统设计	(约) 6小时	(约) 3小时
编码调试	(约) 20小时	(约) 12小时
测试设计	(约) 30分钟	(约) 2分钟
结果分析	(约) 4小时	(约) 2小时

十三、软件过程剪裁说明

保留过程

- 需求分析过程
- 设计过程
- 实现过程
- 测试过程
- 管理过程

简化过程

- 文档过程
- 配置管理过程
- 风险管理过程

省略过程

- 产品发布过程
- 软件维护过程

这样既保证了过程完整性，又符合项目实际情况。

十四、AI赋能前后软件过程对比分析

14.1 需求分析阶段对比

传统方式主要依赖人工整理需求，耗时较长且容易遗漏内容。

引入AI后，开发人员只需描述项目目标，即可快速生成需求说明和需求分类结果，提高需求分析效率和完整性。

14.2 系统设计阶段对比

传统方式需要阅读大量文档与源码才能理解系统架构。

AI能够快速解释系统结构、模块关系以及数据流，大幅降低学习成本。

14.3 编码实现阶段对比

传统开发过程中，大量时间消耗在问题排查和环境配置上。

AI能够分析错误日志并提供解决方案，显著缩短调试时间。

14.4 测试阶段对比

传统测试主要依赖开发人员经验。

AI能够自动生成覆盖功能测试、异常测试、性能测试以及兼容性测试的大量测试用例，提高测试覆盖率。

14.5 综合评价

总体来看，AI技术的引入使软件过程从经验驱动逐步向智能辅助驱动转变，提高了开发效率和过程质量。

十五、软件过程持续改进机制

15.1 经验反馈机制

记录项目实施过程中遇到的问题和经验。

包括：

- 编译问题；
- 环境配置问题；
- 测试缺陷；
- AI使用经验。

15.2 AI提示词优化机制

提示词质量直接影响AI输出质量。

通过不断优化提示词，提高AI辅助开发效果。

15.3 过程评估机制

项目结束后，对软件过程进行评估。

评估内容包括：

- 开发效率；
- 缺陷发现率；
- 测试覆盖率；
- AI使用效果。

15.4 知识库建设

建立项目知识库，积累：

- 技术文档；
- 常见问题；
- 测试案例；
- AI提示词模板。

十六、总结

本文基于ISO/IEC 12207软件生命周期过程模型，结合生成式人工智能技术，对ORB-SLAM3单目SLAM系统部署与实验项目的软件过程进行了全面改进。

通过引入ChatGPT、Cursor等AI工具，在需求分析、系统设计、编码实现、测试验证、结果分析以及项目管理等环节建立了AI辅助机制，形成了“人工决策、AI辅助、人机协同”的软件过程模式。

实践分析表明，该过程能够有效降低开发成本、提高开发效率、增强测试覆盖率，并提升项目整体质量。